

Git and GitHub: A Practical Overview of Version Control Workflows

RamaKrishna

Abstract—Git is a distributed version control system that allows developers to track changes to source code and collaborate on shared projects. This paper summarizes the core Git and GitHub workflow, including repository initialization, staging and committing changes, branching, merging, rebasing, and history manipulation using `reset` and `revert`. The material presented is based on practical exercises completed using the interactive tool Learn Git Branching, and is intended as a concise reference for students learning version control for the first time.

Index Terms—Git, GitHub, version control, branching, merging, rebase, commit history.

I. INTRODUCTION

Git is a version control system used to track changes in files, particularly source code. It is *distributed*, meaning multiple contributors can work on the same project simultaneously using their own local repositories, without depending on a single central copy of the code.

GitHub is a cloud-based platform for hosting and collaborating on code using Git. It provides a central location where developers store their code, track its history, and coordinate work with others.

A. Repositories

A repository (“repo”) is analogous to a project folder. It contains a project’s files together with the complete history of changes made to them. Repositories may be:

- 1) **Public** — accessible to anyone.
- 2) **Private** — accessible only to invited collaborators.

B. Motivation for Using Git

Git is used primarily because it (1) tracks every change made to a project over time, and (2) facilitates collaboration among multiple developers working on the same codebase.

II. CORE CONCEPTS

A. Commits

A *commit* is a saved snapshot of a project’s tracked files at a given point in time. Commits act as checkpoints that a developer can always return to. With the exception of the first commit in a repository, every commit has a *parent* commit, which together form the project’s history.

B. Staging Changes

The command `git add` specifies which changes should be included in the next commit. The standard local workflow therefore follows a fixed sequence:

edit → add → commit

III. BRANCHING

Branching creates an independent line of development, allowing a developer to work on a feature or bug fix without affecting the main codebase. Internally, a branch is simply a pointer to a specific commit.

Table I. Common Branch Commands

Command	Purpose
<code>git branch</code>	List branches / show current branch
<code>git branch -m new_name</code>	Rename current branch
<code>git checkout branch_name</code>	Switch to an existing branch
<code>git checkout -b new_branch</code>	Create and switch to a new branch
<code>git branch -d branch_name</code>	Delete a branch

Table II. File States Reported by `git status`

State	Description
Untracked	New file not yet tracked by Git
Modified	Changed, but not yet staged
Staged	Added and ready to be committed
Unchanged	No pending changes (post-commit)

```
# Create and switch to a new branch
git branch newname # Step 1: create branch
git checkout newname # Step 2: switch to branch
git commit # Step 3: commit on branch
```

Table I summarizes the most frequently used branch commands.

IV. CLONING A REPOSITORY

Cloning creates a complete local copy of a remote repository (typically hosted on GitHub):

Remote (GitHub) → Local (developer machine)

```
git clone <repository-link>
```

V. REPOSITORY STATUS

The command `git status` reports the current state of the working directory relative to the repository. Files may be in one of four states, summarized in Table II.

VI. INITIALIZING A REPOSITORY

A new, empty Git repository is created with:

```
git init
```

VII. PUSHING CHANGES

Pushing uploads local commits to a remote repository such as GitHub.

```
git remote add origin <link> # connect to remote (once)
git branch # confirm current branch
git push origin main # upload commits
```

By convention, `origin` is the default name Git assigns to the remote repository from which a project was cloned. The overall local-to-remote workflow can be summarized as:

clone → edit → add → commit → push

VIII. MERGING

Merging combines the changes of one branch into another, allowing a feature to be developed in isolation and later integrated into the main line of development. A merge produces a special commit with two parent commits.

```
git checkout <branch-1>
git merge <branch-2>
```

A. Merging via Pull Request

A *pull request* (PR) proposes that the changes on one branch be merged into another, enabling peer review before integration.

```
git checkout -b <new-branch>
# edit files
git add .
git commit -m "added changes"
git push origin <new-branch>
```

The resulting review workflow on GitHub is:

feature branch → PR → review → approve → merge → main

After merging, the local main branch is updated with:

```
git checkout main
git pull origin main
```

B. Resolving Merge Conflicts

A merge conflict arises when Git cannot automatically reconcile changes made to the same part of a file on two different branches.

```
git checkout main
git pull origin main
git checkout feature
git merge main
# CONFLICT occurs

git status
# remove conflict markers, keep correct code
git add .
git commit -m "Resolve merge conflict"
git push origin feature
```

IX. GIT REBASE

Rebasing is an alternative to merging that moves a branch's commits onto the tip of another branch, producing a linear history that appears sequential rather than parallel.

```
git checkout bugFix
git rebase main
```

Table III. Rebase-Related Commands

Command	Purpose
<code>git rebase main</code>	Rebase current branch onto main
<code>git status</code>	Check rebase / conflict status
<code>git add .</code>	Mark conflicts as resolved
<code>git rebase -continue</code>	Continue the rebase
<code>git rebase -abort</code>	Cancel the rebase

X. NAVIGATING REPOSITORY HISTORY

A. HEAD

HEAD is a pointer indicating the commit currently checked out. Ordinarily it points to a branch, but it may also be *detached*, pointing directly to a commit:

```
git checkout <commit-hash> # HEAD detaches
```

B. Relative References

Relative references specify a commit relative to another (usually HEAD) without requiring the full commit hash. The caret (^) moves up one commit; ~<num> moves up multiple commits.

```
git checkout HEAD~2
```

C. Branch Forcing

Branch forcing moves a branch pointer to a specified commit even when Git would normally disallow the operation, typically to correct or reorganize history.

```
git branch -f main HEAD~3
```

XI. RESET AND REVERT

Both `git reset` and `git revert` undo changes, but they operate very differently and are suited to different situations, as summarized in Table IV.

A. Reset

`git reset` moves the current branch pointer backward to an earlier commit, effectively discarding later commits from that branch's history. It is best suited to local, unshared history.

```
git reset B
```

$A-B-C-D$ (HEAD at D) $\longrightarrow$ $A-B$ (HEAD at B)

B. Revert

`git revert` undoes the changes introduced by a commit by creating a *new* commit, leaving the original commit intact. It is the safer option for commits that have already been pushed and shared with others.

```
git revert C
```

$A-B-C \longrightarrow A-B-C-C'$ (C' undoes C)

Table IV. Comparison of `git reset` and `git revert`

git reset	git revert
Moves branch backward	Creates a new commit
Can rewrite history	Preserves history
Good for local changes	Good for shared/pushed changes
Old commit can disappear	Old commit remains in history

XII. INTERACTIVE REBASE

Interactive rebase allows commits to be edited, reordered, combined, or deleted before they become part of shared history:

```
git rebase -i HEAD~4
```

Within the interactive view, commits may be reordered by dragging, and individual commits may be dropped by disabling their “pick” option.

XIII. STACKED COMMITS AND COMMIT JUGGLING

Locally stacked commits are successive commits on a local branch, each built on the previous one, that have not yet been pushed to the remote repository. *Commit juggling* refers to rearranging or transferring such commits between branches to

achieve a desired history — for example, reordering commits with interactive rebase so that an older commit can be modified, then restoring the original order.

XIV. TAGS

A Git *tag* assigns a permanent name to a specific commit, typically used to mark releases or other significant milestones. Unlike branches, tags do not move as new commits are added, and a tag cannot be checked out for direct editing.

The command `git describe` reports the current position relative to the nearest tag:

```
git describe <ref>
```

producing output of the form
`<tag>-<commits-since-tag>-g<hash>`.

XV. CONCLUSION

Git provides a robust framework for tracking changes and coordinating collaborative software development, while GitHub extends this functionality with remote hosting, pull requests, and code review. Mastery of the core commands presented here — `add`, `commit`, `branch`, `merge`, `rebase`, `reset`, and `revert` — provides a practical foundation for managing real-world software projects.